

08

/ 13 LECTURES

Routing and Forwarding — Part II

Distance Vector and RIP, count-to-infinity and its cures, Link State and OSPF, BGP between Autonomous Systems, convergence and routing loops.

Distance Vector

RIP

Count-to-Infinity

Split Horizon

Poison Reverse

Link State · OSPF

LSA · LSDB

BGP & AS

Convergence

Routing Loops

WHAT THIS LECTURE GIVES YOU

How routers LEARN routes without a human: distance-vector tables converging round by round, RIP and the classic count-to-infinity failure with all four fixes, OSPF's flood-and-Dijkstra machine with cost calculation and areas, and BGP — the protocol that glues the world's Autonomous Systems, including how hijacks happen.

LEARNING OBJECTIVES — AFTER THIS LECTURE YOU CAN

- Trace **distance-vector** routing convergence across 4 routers
- Explain **count-to-infinity** and all four solutions
- Describe **OSPF** operation: Hello, LSAs, LSDB, SPF, areas
- Explain **BGP** conceptually: AS paths, route selection, hijacking
- Define **convergence** and **routing loops**, with prevention techniques

Routing II

Distance Vector

share whole table to neighbours

distributed Bellman-Ford

→ RIP (15-hop limit)

Count-to-∞

split horizon

poison reverse

triggered updates

holddown timer

Link State · OSPF

Hello → LSA flood → LSDB

everyone runs Dijkstra

cost = 100 Mbps / BW

areas (backbone 0)

BGP

between ASes (EGP)

AS_PATH loop prevention

policy-driven

hijack via longer prefix

Convergence

RIP: 30s–minutes

OSPF: 1–10 s

BGP: minutes–hours

1 Distance Vector Routing

Every router: (1) knows costs to **direct neighbours**, (2) periodically sends its **entire routing table** to those neighbours, (3) updates using the Bellman-Ford rule:

$$\text{dist}[\text{dest}] = \min(\text{dist}[\text{dest}], \text{cost_to_neighbor} + \text{neighbor_dist}[\text{dest}])$$

distributed Bellman-Ford — "tell me your distances, I'll add my cost to reach you"

Convergence trace — line topology A—B—C—D (all links cost 1)

State	A's table	B's table	C's table	D's table
Initial (direct links only)	B:1, C:∞, D:∞	A:1, C:1, D:∞	B:1, D:1, A:∞	C:1, A:∞, B:∞
After round 1	B:1, C:2, D:∞	A:1, C:1, D:2	B:1, D:1, A:2	C:1, B:2, A:∞
After round 2 — converged	B:1, C:2, D:3	A:1, C:1, D:2	B:1, D:1, A:2	C:1, B:2, A:3

Each round, information travels one more hop — good news spreads at "one hop per round" speed.

RIP — Routing Information Protocol

Property	Value	Property	Value
Metric	Hop count	Update interval	every 30 s
Max hops	15 (16 = unreachable)	Algorithm	distributed Bellman-Ford
RIPv1 → RIPv2 adds	CIDR masks, authentication	Scope	small networks only

2 Count-to-Infinity — and the Four Fixes

Setup: A — B — C in a line. Normal state: B reaches C directly (1), A reaches C via B (2). Now the **B–C link fails**:

The failure (bad news travels slowly)

```
B detects failure → sets C = ∞
A's old table arrives: "C = 2 via me..."
B thinks: via A → C = 1+2 = 3 ← WRONG (A's path went through B!)
B tells A: C = 3 → A updates: 4
A tells B: 4 → B updates: 5 ...
...creeps up to 16 = unreachable (RIP's ∞)
```

Why it's absurd

A's route to C goes *through B* — so A can never offer B a shortcut. The routers keep "discovering" routes that route through each other: a **transient routing loop** where packets bounce A↔B until TTL kills them, and convergence takes minutes.

Fix	Rule	Effect
1. Split Horizon	Never advertise a route back on the interface you learned it from	A never tells B "I can reach C" — the loop never starts
2. Poison Reverse	Instead of silence: advertise the route back as ∞ (16)	Stronger — B pre-knows A has no independent path; instant convergence
3. Triggered Updates	On failure: flood "C = 16" immediately , don't wait 30 s	Shrinks the window where stale info circulates
4. Holddown Timer	After poisoning, ignore claims that the route is reachable for N seconds	Prevents a slow neighbour's stale table from resurrecting the dead route

EXAM The 8-mark version: "Explain count-to-infinity with a diagram and state its solutions." Draw A—B—C, show the failed link, walk the 3→4→5 numbers, then the four fixes with one-line mechanisms. This exact narrative is what graders look for.

3 Link State Routing — OSPF

Idea: make every router see the **same complete map**, then let each run Dijkstra independently — no more neighbour-gossip.

1 Discover neighbours — Hello packets (every 10 s)

Agree on hello/dead intervals, area ID, authentication → become neighbours.

2 Build a Link State Advertisement (LSA)

"I am R1. Link to R2 costs 10, link to R3 costs 5."

3 Flood LSAs to ALL routers in the area

Every router assembles the same Link State Database (LSDB) — the full topology map.

4 Run Dijkstra (SPF) on the LSDB

Each router computes shortest paths from itself → identical conclusions → consistent, loop-free tables.

5 React to change

Link fails → new LSA floods immediately → SPF re-runs. Convergence in seconds (Lecture 6's LS table explains why this beats DV).

OSPF cost formula

$$\text{cost} = \text{ref_bw} / \text{iface_bw}$$

ref = 100 Mbps default → 100M:1 · 10M:10 · 1M:100
— slower links cost more

Areas — scaling OSPF

Big networks split into **areas**; LSAs flood *within* an area only. All areas attach to **Area 0 (backbone)**; ABRs (Area Border Routers) summarize between areas.

4 BGP — the Protocol Between Organisations

- An **AS (Autonomous System)** = one organisation's network with a unique ASN: Google AS15169, Cloudflare AS13335, Jio AS55836, BSNL AS9829.
- **BGP (Border Gateway Protocol)** = the EGP exchanging routes **between** ASes. It's **policy-driven**: business contracts decide paths as much as distance.
- **Path-vector**: each advertisement carries the full **AS_PATH** list of networks traversed:

```
Google announces:      8.8.8.0/24   AS_PATH: [15169]
Tata re-advertises:    8.8.8.0/24   AS_PATH: [6453, 15169]
Airtel re-advertises:  8.8.8.0/24   AS_PATH: [9498, 6453, 15169]
# Loop prevention: your own ASN already in the path? → discard the route.
```

BGP best-path order (simplified)

Meaning

- | | |
|----------------------------------|--|
| 1. Highest LOCAL_PREF | Your own policy preference (business choice) |
| 2. Shortest AS_PATH | Fewer organisations crossed |
| 3. Lowest MED | Neighbour's hint about which entry point to prefer |
| 4. eBGP over iBGP | External routes preferred over internal |
| 5. Lowest IGP metric to next hop | Closer exit wins ties |

eBGP vs iBGP

eBGP runs between different ASes (the Internet's stitching). **iBGP** spreads externally-learned routes *inside* your own AS so all border routers agree.

BGP hijacking (real world)

Announce someone else's prefix as **more specific** (a /24 beats their /16 — LPM again!). 2008: Pakistan Telecom announced YouTube's /24 → worldwide YouTube traffic landed in Pakistan. Defence: **RPKI** cryptographic route signing.

5 Convergence & Routing Loops

Convergence = all routers holding consistent, correct tables. **Convergence time** = how long after a change before that's true.

Protocol	Convergence	Why
RIP	30 s – minutes	30 s timers + count-to-infinity risk
OSPF	1–10 s	triggered LSA flooding + instant SPF
BGP	minutes – hours	policy processing, route dampening, sheer scale

Routing loop = packets circling forever

```
A's table: 10.1.0.0/24 → via B      B's table: 10.1.0.0/24 → via A  ← LOOP!
packet → A → B → A → B → ... until TTL=0 → dropped + ICMP Time Exceeded
```

Loop prevention technique	Protocol	Mechanism
TTL	IP (universal safety net)	Packets die after N hops
Split horizon / poison reverse	RIP	Don't feed routes back to their source
AS_PATH check	BGP	Own ASN in path → discard
Full topology	OSPF	Same map + same algorithm = consistent tables
STP	Ethernet (L2)	Block redundant switch links

The big comparison (memorise)

Feature	RIP	OSPF	BGP
Type	Distance Vector	Link State	Path Vector
Algorithm	Bellman-Ford	Dijkstra (SPF)	Policy-based selection
Metric	Hop count	Cost (bandwidth)	Policy attributes
Scope	Inside AS (IGP)	Inside AS (IGP)	Between ASes (EGP)
Max size	~15 hops	Large (with areas)	Internet-scale
Convergence	Slow	Fast	Slow (deliberate)
Loop prevention	Split horizon	Full topology	AS_PATH
Standard	RFC 2453	RFC 2328	RFC 4271

EXAM

Classic: "Two paths exist — 3 hops over 10 Mbps vs 5 hops over 1 Gbps. Which wins in RIP? In OSPF?" RIP picks **3 hops** (metric = hops). OSPF picks the **1 Gbps** path (costs 1+1+1+1+1=5 vs 3×10=30 — bandwidth matters, hops don't).

LECTURE 8 — RAPID REVISION CARD

DV: share whole table with neighbours; $\text{dist} = \min(\text{mine}, \text{mine-to-nbr} + \text{nbr's})$.

RIP: hop metric, max 15, 30 s updates, Bellman-Ford.

Count-to- ∞ : failed route resurrected via the neighbour who learned it from you. Fixes: split horizon · poison reverse · triggered updates · holddown.

OSPF: Hello → LSA flood → common LSDB → each runs Dijkstra → fast, loop-free.

OSPF cost = 100 Mbps / BW; areas hang off backbone Area 0.

BGP: between ASes; policy-driven; AS_PATH = loop prevention; eBGP vs iBGP.

Hijack = announcing a more-specific prefix of someone else (LPM abuse); RPKI defends.

Loops: TTL is the universal net; RIP/OSPF/BGP each prevent loops their own way.

✦ Important Questions — Exam Practice

A. Multiple Choice

Q1. In RIP, a metric of 16 means:

MCQ

- a) 16 hops away
- b) unreachable / infinity
- c) best route
- d) load balance across 16 paths

Q2. Split horizon says:

MCQ

- a) share tables only at midnight
- b) never advertise a route back to the interface it was learned on
- c) use two routers for backup
- d) poison all routes

Q3. The OSPF database holding the full topology map is the:

MCQ

- a) RIB
- b) LSDB
- c) ARP cache
- d) NAT table

Q4. BGP prevents loops using:

MCQ

- a) TTL
- b) split horizon
- c) AS_PATH containing own ASN → discard
- d) Dijkstra

Q5. OSPF cost of a 10 Mbps interface (ref 100 Mbps) is:

MCQ

- a) 1
- b) 10
- c) 100
- d) 0.1

B. Short Answer (2–4 marks)

Q6. Trace DV convergence for A—B—C—D from D's perspective (how does D learn about A?).

SHORT

Q7. Why does poison reverse converge faster than plain split horizon?

SHORT

Q8. List the five OSPF operation steps in order.

SHORT

Q9. What is an Autonomous System? Give two real ASNs and their owners.

SHORT

Q10. Define convergence. Rank RIP, OSPF, BGP by convergence speed with reasons.

SHORT

C. Long Answer (5–8 marks)

Q11. Explain count-to-infinity with the A—B—C diagram, walk through the numbers after the B—C failure, and describe all four solutions precisely.

LONG

Q12. Describe OSPF operation end-to-end (Hello → LSA → LSDB → SPF → install), the cost formula with three computed examples, and why areas exist.

LONG

Q13. Compare RIP, OSPF and BGP in a table (type, algorithm, metric, scope, size, convergence, loop prevention). Which would you deploy: inside a 40-router campus, inside a 3-router lab, between two ISPs?

LONG

Q14. Explain BGP path-vector operation, the simplified best-path order, and how the 2008 YouTube hijack worked — connecting it to Longest Prefix Match from Lecture 7.

LONG

Q15. Why is BGP called path-vector rather than distance-vector?

SHORT

ANSWER KEY

A1. b — 16 is RIP's infinity.

A2. b — routes learned on interface X are never advertised back out of X.

A3. b — Link State Database; Dijkstra runs on it.

A4. c — seeing your own ASN means the route already visited you.

A5. b — $100/10 = 10$.

A6. Round 1: C learns A:2 from B → tells D. Round 2: D records A:3 via C. Info advances one hop per round.

A7. Split horizon only withholds; poison reverse actively states "∞", so the neighbour's table never holds a stale usable route during the transition.

A8. Hello/discovery → adjacency/LSA exchange → LSA flooding → SPF (Dijkstra) → route installation.

A9. One administration's network with an ASN — Google AS15169, Cloudflare AS13335 (also Jio AS55836, BSNL AS9829).

A10. All routers agree on tables. OSPF (1–10 s, triggered flooding) > RIP (30 s–minutes, timers) — BGP slowest by design (policy, scale).

A13. Table as in §5. Campus → OSPF; 3-router lab → RIP (or static); ISPs → BGP.

A14. Path = full AS list rather than a scalar distance; policy attributes dominate; hijack = announcing a /24 inside the victim's /16 — more specific wins LPM → traffic misrouted; RPKI signs announcements.

A15. It carries the entire AS_PATH (a vector of ASes), not just a hop/cost number — enabling policy + loop detection by inspection.